Proyecto Integrador Académico (PIA) – Compiladores AD2026

Introducción

El presente Proyecto Integrador Académico (PIA) tiene como finalidad aplicar de manera progresiva los conceptos fundamentales y avanzados estudiados en la materia de Compiladores, mediante el diseño e implementación de un compilador básico o sistema de traducción de lenguaje.

A lo largo del proyecto, el estudiante desarrollará un proceso completo que abarque desde el análisis léxico hasta la generación de código intermedio o final, integrando conocimientos de lenguajes formales, estructuras de datos, análisis sintáctico, análisis semántico y optimización de código.

El enfoque principal no se limita a la implementación técnica, sino que también enfatiza la comprensión del funcionamiento interno de los compiladores, la justificación de decisiones de diseño y la correcta interpretación del proceso de traducción de programas.

Este proyecto fomenta el pensamiento estructurado, la abstracción, la capacidad de modelar lenguajes y el desarrollo de software de sistemas, además de promover el trabajo en equipo y la documentación técnica.

Objetivo General

Aplicar los conceptos fundamentales de compiladores mediante el diseño, implementación y validación de un compilador básico o sistema de traducción, desde el análisis léxico hasta la generación de código.

Objetivos Específicos

- Diseñar un lenguaje o subconjunto de lenguaje a compilar
- Implementar un analizador léxico basado en tokens y patrones
- Construir un analizador sintáctico utilizando gramáticas formales
- Generar un Árbol de Sintaxis Abstracta (AST)
- Implementar una tabla de símbolos y análisis semántico
- Generar código intermedio (por ejemplo, código de tres direcciones)
- Aplicar optimizaciones básicas al código generado
- Evaluar y validar el funcionamiento del compilador
- Documentar el proceso completo de desarrollo

Selección del Proyecto

Cada equipo deberá seleccionar el tipo de compilador o sistema a desarrollar. Algunas opciones sugeridas incluyen:

- Compilador de un lenguaje simple (expresiones, variables, control de flujo)
- Intérprete de lenguaje personalizado
- Traductor de un lenguaje a otro (por ejemplo, pseudo código a Python)
- Evaluador de expresiones matemáticas
- Compilador de un lenguaje tipo calculadora con funciones
- Mini compilador educativo con AST y ejecución
- Analizador estático de código

El proyecto deberá ser aprobado por el docente.

Temas Aplicados en el Proyecto

El proyecto deberá integrar los siguientes temas del curso:

1. Análisis léxico (tokens, expresiones regulares)
2. Análisis sintáctico (gramáticas libres de contexto)
3. Árboles de derivación y AST
4. Análisis semántico
5. Tabla de símbolos
6. Alcance y tipos
7. Generación de código intermedio (TAC)
8. Representación del flujo de control
9. Forma SSA (opcional)
10. Optimización de código
11. Generación de código final (opcional)
12. Manejo de errores

Requisitos del Proyecto

El sistema desarrollado deberá:

- Procesar correctamente un lenguaje definido
- Detectar errores léxicos, sintácticos y semánticos

• Generar una representación estructurada (AST o IR)
• Incluir al menos una etapa de generación de código intermedio
• Permitir ejecutar o evaluar el programa (si aplica)
• Incluir pruebas de funcionamiento
• Contar con documentación clara

Estructura del Proyecto

Semana 1: Definición del Lenguaje y Planeación

• Definición del lenguaje a implementar
• Gramática preliminar
• Identificación de tokens
• Ejemplos de entrada válidos e inválidos
• Alcance del proyecto
• Cronograma y roles del equipo
• Firma de integridad académica

Semana 2: Análisis Léxico

• Implementación del analizador léxico
• Definición de tokens, lexemas y patrones
• Uso de expresiones regulares
• Manejo de errores léxicos
• Pruebas de tokenización
• Documentación del scanner

Semana 3: Análisis Sintáctico

• Definición de gramática formal
• Implementación del parser (LL o LR)
• Construcción del árbol de derivación
• Generación del AST
• Manejo de errores sintácticos
• Validación de estructura del lenguaje

Semana 4: Análisis Semántico

• Implementación de tabla de símbolos
• Manejo de alcance y visibilidad
• Verificación de tipos
• Detección de errores semánticos

• Anotación del AST
• Validación del significado del programa

Semana 5: Código Intermedio y Optimización

• Generación de código de tres direcciones (TAC)
• Representación de expresiones
• Implementación de estructuras de control
• Introducción a SSA (opcional)
• Aplicación de optimizaciones básicas:

- Propagación de constantes
- Eliminación de código muerto
  • Validación del código generado

Presentación Final

• Versión final del compilador
• Pruebas completas del sistema
• Ejecución de ejemplos
• Documentación completa
• Video explicativo (5 minutos)
• Exposición del proyecto

Entregables Generales

• Documento del proyecto (PDF o Word)
• Código fuente documentado
• Definición del lenguaje
• Ejemplos de entrada y salida
• AST o IR generada
• Resultados de ejecución
• Video explicativo
• Cronograma
• Declaración de originalidad

Criterios de Evaluación

Criterio	Porcentaje
Diseño del lenguaje y gramática	15%
Implementación del compilador	25%

Análisis semántico y validación	20%
Generación de código y optimización	20%
Documentación y presentación	10%
Cumplimiento y organización	10%

Integridad Académica

Todos los modelos, códigos y análisis deben ser originales.
El uso de fuentes externas debe ser referenciado adecuadamente.
El plagio o uso indebido de código será sancionado conforme al reglamento institucional.

Nota aclaratoria sobre la evaluación de entregables

Para efectos de organización y evaluación del Proyecto Integrador Académico, los entregables se agruparán de la siguiente manera:

• Los contenidos correspondientes a las Semana 1 y 2 serán evaluados como **Entregable 1**
• Los contenidos correspondientes a las Semana 3, 4 y 5 serán evaluados como **Entregable 2**

Esta estructura permite mantener coherencia con las etapas del desarrollo del compilador, desde el análisis hasta la generación de código.